

Guia Prático de Git

Como enviar alterações de arquivos locais do seu computador para o repositório remoto

FLUXO DE TRABALHO

COMANDOS ESSENCIAIS

GITHUB / GITLAB

TERMINAL

1. Entendendo o Fluxo de Trabalho do Git

Quando você altera um arquivo na sua máquina, o Git não envia a mudança automaticamente para a nuvem. O processo ocorre em 4 áreas fundamentais:

ETAPA 1

Working Tree

Sua pasta local. Arquivos editados ficam marcados como *modificados*.

ETAPA 2

Staging Area

A "sala de espera". Onde você escolhe quais arquivos vão no próximo pacote.

ETAPA 3

Local Repository

Histórico no seu PC. O *commit* registra a versão localmente no Git.

ETAPA 4

Remote Repo

Nuvem (GitHub/GitLab). O *push* envia os commits salvos para o servidor.

2. Passo a Passo Completo para Enviar Suas Alterações

1 Verificar quais arquivos foram alterados

Abra o terminal na pasta do projeto e verifique o estado atual dos seus arquivos:

```
$ git status
# Exibe arquivos modificados, novos (untracked) ou deletados
```

O que esperar: Os arquivos alterados aparecerão listados em vermelho (não adicionados à Staging Area).

2 (Opcional) Conferir as mudanças exatas nos arquivos

Para inspecionar exatamente o que foi adicionado ou removido linha por linha antes de salvar:

```
$ git diff
# Exibe as linhas adicionadas (+) e removidas (-) nos arquivos
```

3 Adicionar os arquivos à Área de Preparação (Staging Area)

Mova os arquivos desejados para a "zona de preparação". Arquivos adicionados ficam em verde no `git status`.

Para adicionar **TODOS** os arquivos modificados e novos:

```
$ git add .
```

Para adicionar apenas um arquivo específico:

```
$ git add caminho/do/arquivo.ext
```

4 Criar o Commit (Salvar snapshot no Repositório Local)

O *commit* grava permanentemente o pacote de alterações no histórico da sua máquina local com uma mensagem explicativa.

```
$ git commit -m "feat: atualiza layout do painel e corrige estilos"
```

Boas Práticas de Mensagens de Commit:

Escreva mensagens claras e diretas usando verbos no imperativo/presente: `fix: corrige bug de login` | `docs: atualiza documentacao` | `feat: adiciona filtro`.

5 (Recomendado) Sincronizar com o servidor remoto antes do envio

Garante que seu código local está atualizado em relação às mudanças enviadas por outros colaboradores:

```
$ git pull origin main
# Baixa e mescla alterações da nuvem (substitua 'main' pela sua branch ativa)
```

6 Enviar os commits locais para o servidor remoto (Push)

Envia seus commits locais salvos para o repositório hospedado na nuvem (GitHub, GitLab, Bitbucket):

```
$ git push origin main
```

Sucesso!

Suas alterações locais agora estão salvas no servidor remoto e visíveis para toda a sua equipe.

3. Resumo Rápido dos Comandos (Cheat Sheet)

Comando	Etapa	Descrição do Efeito
<code>git status</code>	Verificação	Lista quais arquivos foram alterados, criados ou excluídos na pasta local.
<code>git diff</code>	Inspeção	Exibe as linhas exatas de código alteradas nos arquivos.
<code>git add .</code>	Staging Area	Prepara todas as alterações para o próximo commit.
<code>git add arquivo.ext</code>	Staging Area	Prepara apenas um arquivo individual.
<code>git commit -m "msg"</code>	Repositório Local	Salva as alterações no histórico do seu PC com uma mensagem descritiva.
<code>git pull origin main</code>	Sincronização	Baixa novidades da nuvem para o seu PC antes do envio das suas alterações.
<code>git push origin main</code>	Repositório Remoto	Transfere os commits do seu computador para o servidor (GitHub/ GitLab).

4. Resolução de Situações Comuns (Troubleshooting)

A. Como desfazer um `git add` feito por engano?

Para remover um arquivo da Staging Area sem perder suas edições de código:

```
$ git restore --staged nome-do-arquivo.txt
```

B. Como ver o histórico dos commits já realizados?

Para visualizar a lista resumida de commits salvos na sua máquina:

```
$ git log --oneline
```

C. O que fazer se o `git push` for rejeitado?

Geralmente ocorre porque o repositório remoto possui commits que você ainda não baixou.

Solução:

Execute `git pull origin main` para baixar os commits mais recentes. Se houver conflitos de mesclagem, abra os arquivos no seu editor, resolva os trechos sinalizados pelo Git, depois execute `git add .`, `git commit -m "fix: resolve conflitos"` e tente o `git push origin main` novamente.

D. Como criar e enviar uma nova Branch (Ramificação)?

Trabalhar em ramificações separadas evita interferir na versão principal (`main`):

```
$ git checkout -b minha-nova-feature
# Cria e altera imediatamente para a nova branch

$ git push -u origin minha-nova-feature
# Vincula e envia a nova branch para o servidor remoto
```

5. Boas Práticas Essenciais ao Trabalhar com Git

- **Use o arquivo `.gitignore`:** Sempre configure o `.gitignore` para evitar enviar senhas, arquivos de ambiente (`.env`), dependências geradas (`node_modules/`, `venv/`) ou pastas do sistema.
- **Commits pequenos e frequentes:** Não acumule dias de trabalho em um único commit imenso. Faça commits lógicos a cada pequena funcionalidade ou correção concluída.
- **Verifique sempre antes de commitar:** Crie o hábito de executar `git status` antes do `git add` para garantir que está enviando exatamente os arquivos desejados.